Laxu Litepaper

Make your position capital efficient.

Version 0.1 · October 2026 · Monad testnet (chain ID 10143) · Trading on Perpl

> Testnet only, no real funds. Laxu is an independent project built on Perpl, not affiliated with or endorsed by Perpl.

Contents: 1. Abstract · 2. The problem · 3. How Laxu works · 4. Mechanism design · 5. The invariants we hold · 6. Trust model and tradeoffs · 7. Decisions and what we cut · 8. Market and positioning · 9. Go-to-market · 10. Business model · 11. Roadmap · 12. Risks and disclosures · 13. Credits and how it was built · Appendix

---

1. Abstract

Laxu turns each open perp trade on Perpl into its own token on Monad, and gives every token an isolated lending pool. Holders borrow **AUSD** against the live value of their trade while it keeps running. Once the creator lists a trade, others can buy into it, and the creator earns 2% of each buy-in.

The token prices itself on-chain: it reads the Perpl mark price from Perpl's exchange contract on every valuation, and adds cumulative funding pushed by the Laxu backend. The lending pool reads that value live, so a winning trade raises its own borrowing limit without anyone sending a transaction.

Borrowers with a Privy embedded wallet can turn on **loan protection**: a Privy signer, bounded by a policy that allows only `repay` on that one pool, repays part of the loan from the user's own wallet when its health falls to a level the user chose. It can do nothing else.

Status: the contracts are deployed on Monad testnet. The core flow (open a real Perpl trade, receive the token, borrow against it, repay) runs end to end; the recorded run is in `docs/e2e-run.md`.

---

2. The problem

Who has it. Perp traders on Perpl, the perpetual futures exchange on Monad. Its testnet lists eight markets: BTC, ETH, SOL, PUMP, MON, ZEC, LIT and NEAR.

What they do today. A trader with an open, profitable position who needs cash can:

- **close the trade**, giving up the exposure they wanted; or
- **withdraw margin**, shrinking the cushion and raising the effective leverage.

Either way, the value of the trade cannot be used while the trade stays as it is.

Why existing options do not cover it:

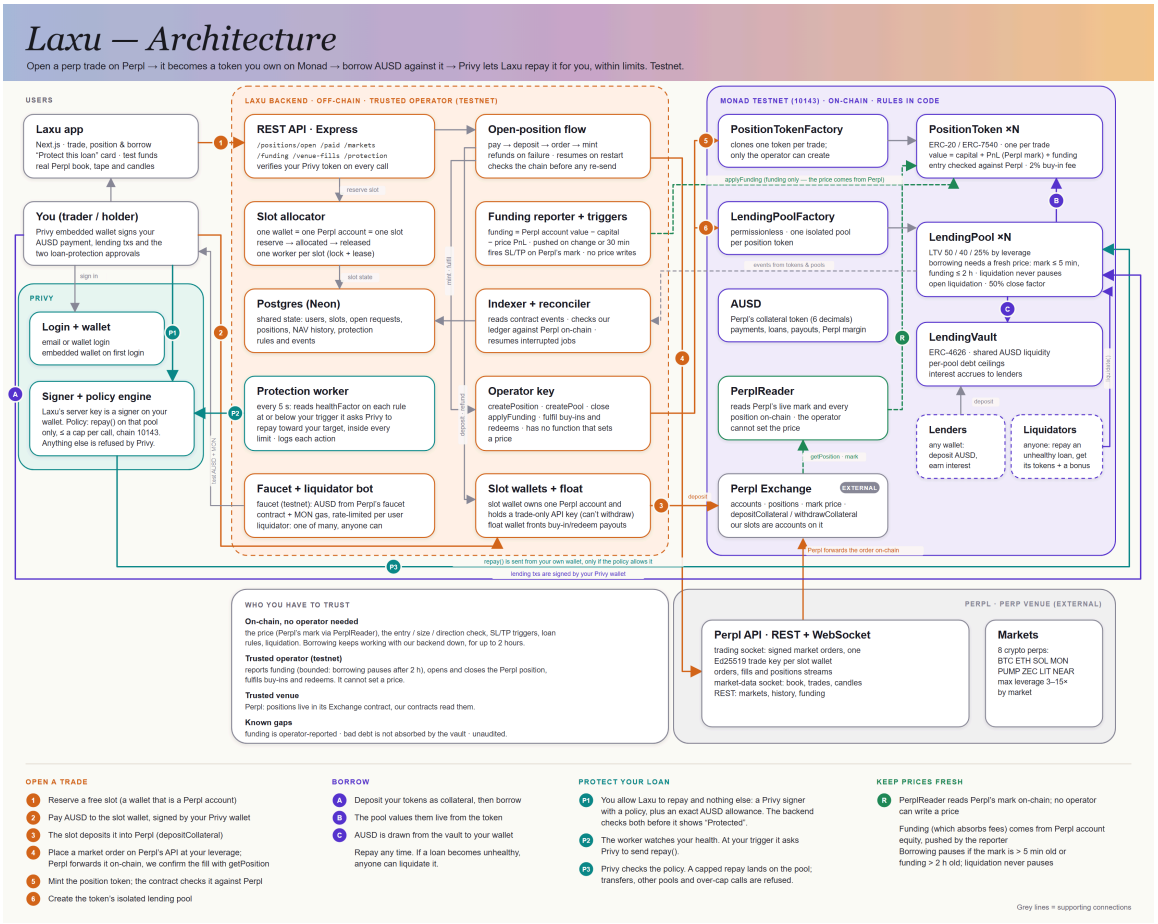
- **Perp venues** do not lend against positions.
- **Lending markets** accept deposits and LP shares, not open positions.
- **Leveraged tokens** are platform-defined, with no lending market.

Why now. Per-trade tokens, live on-chain valuation and second-by-second repayments are routine on Monad's fast, cheap execution.

---

3. How Laxu works

3.1 Architecture



Five zones: **users** (browser, Privy embedded wallet); **Privy** (auth, signers and policy); **the Laxu backend** (open flow, slot allocator, reporter, indexer and reconciler, liquidator bot, protection worker, Postgres); **Monad** (`PerplReader`, the two factories, one `PositionToken` and one `LendingPool` per trade, the shared `LendingVault`); and **Perpl** (exchange contract, REST and WebSocket API).

OPEN A TRADE

1. Reserve a free slot (a wallet that is a Perpl account)
2. Pay AUSD to the slot wallet, signed by your Privy wallet
3. The slot deposits it into Perpl (depositCollateral)
4. Place a market order on Perpl's API at your leverage; Perpl forwards it on-chain, we confirm the fill with getPosition
5. Mint the position token; the contract checks it against Perpl
6. Create the token's isolated lending pool

BORROW

- A. Deposit your tokens as collateral, then borrow
- B. The pool values them live from the token
- C. AUSD is drawn from the vault to your wallet

Repay any time. If a loan becomes unhealthy, anyone can liquidate it.

PROTECT YOUR LOAN

- P1. You allow Laxu to repay and nothing else: a Privy signer with a policy, plus an exact AUSD allowance. The backend checks both before it shows "Protected".
- P2. The worker watches your health. At your trigger it asks Privy to send repay().
- P3. Privy checks the policy. A capped repay lands on the pool; transfers, other pools and over-cap calls are refused.

KEEP PRICES FRESH

- R. PerplReader reads Perpl's mark on-chain; no operator can write a price

Funding (which absorbs fees) comes from Perpl account equity, pushed by the reporter

Borrowing pauses if the mark is > 5 min old or funding > 2 h old; liquidation never pauses

3.2 Opening a trade

1. **Reserve a slot** (§3.4) for 15 minutes, before any money moves.
2. **Pay.** The user sends the exact AUSD amount to the slot wallet; the backend checks sender, recipient and amount in the receipt.
3. **Deposit.** `depositCollateral` on Perpl's exchange, credited in the same transaction.
4. **Trade.** The backend sends an immediate-or-cancel market order over the slot's WebSocket, with leverage as `lv` (leverage × 100), a strictly increasing request id `rq`, and a limit at the slippage bound. Perpl answers with order (24), fill (25) and position (27) messages.
5. **Mint.** `PositionTokenFactory.createPosition`, with entry and size read from Perpl's contract; the token re-checks them (§4.1).

6. **Pool.** `LendingPoolFactory.createPool` creates the pool and registers it with the vault.

Recorded run: a $50 ETH long at 3× minted 152 s after payment, including one restart (an upper bound).

3.3 Borrowing

- **A.** The user deposits position tokens into the token's `LendingPool`.
- **B.** The pool values them live from the token, which moves with the Perpl mark.
- **C.** AUSD is drawn from the shared `LendingVault` to the user, who can repay at any time.

Worked example (1–5× tier: 50% max LTV, 60% liquidation threshold; fees and funding ignored):

- Open **200 AUSD at 5×** (1,000 notional). The user can borrow 200 × 50% = **100 AUSD**.
- The position gains **+50**. It is worth 250, and the limit rises to 250 × 50% = **125 AUSD** with no transaction.
- With 100 borrowed, health = collateral × 60% / 100 falls below 1 when collateral is below 100 / 0.6 ≈ **166.7**: a loss of 33.3 on 1,000 notional, a **3.3% adverse move** at 5×. Anyone can then liquidate part of the loan.

3.4 What the open flow has to get right

Two systems, one action. Payment, minting and the pool happen on Monad; the deposit and order happen on Perpl. A crash between any two steps must not lose or double-spend money. The flow is a resumable state machine:

```
awaiting_payment → payment_received → deposited → order_filled → minted
       └─ any failure before the fill ─→ refunding → refunded
awaiting_payment ─(reservation expired)─→ failed
```

Each step saves what it learned (transaction hashes, credited amount, `rq`, fill) before moving on. A saved hash is checked by its receipt, never sent again. After a restart every unfinished request resumes, and a database lease gives each request one driver at a time.

Any failure before the fill refunds the user: from the slot wallet, or first withdrawn from Perpl if already deposited. Wrong-amount and late payments are refunded. After the fill the trade is open, so the token must exist: `createPosition` is retried, and before each retry the backend asks the factory whether that trade already has a token, so a crash cannot mint two.

The outcome-unknown rule. Sometimes the socket gives no verdict: a timeout, a dropped connection, or an order the testnet accepted and never answered. Perpl's order history lags about 25 seconds, too slow to rely on. So before each send the

backend saves the `rq`, the order's last execution block `lb`, and the position size. Once the chain passes `lb` it reads the position: **lot count changed means filled** by the difference; **unchanged means never placed**, and it cannot execute later. Only then does the backend send again, under a new `rq`, at most **3** attempts in all. It never sends the same order twice.

Perpl constraints that shaped the design (`docs/perpl-findings.md`):

- **One wallet = one account.** So each open trade gets its own **slot**: a separate wallet-backed Perpl account with its own trade-scope Ed25519 API key. Perpl keys cannot withdraw. A wallet holds at most 16 active keys.
- `rq` **must strictly increase** per account.
- **Order forwarding** is enabled on each account (`allowOrderForwarding(true)`), so the exchange submits API orders on-chain.
- **Opening an account needs 100 AUSD**, which stays as each slot's reserve.
- **Max leverage per market** is min(20, `initial_margin` / 100); Perpl silently clamps anything higher. As read on 5 October 2026: BTC 15×, ETH 12×, SOL 10×, PUMP 5×, MON, ZEC, LIT and NEAR 3×. The backend re-reads them every minute.
- **Taker fee** 0.0345%. **Funding interval** 43 minutes on testnet.

Slots move `free → reserved → allocated → free`. Expired reservations return to `free` under the payment check's row lock, so a payment and an expiry never both win. Each slot has a lock. Settled slots are swept to their reserve and reused. **A new open is refused before the user pays when no slot is free.** 5 concurrent positions today, one per provisioned slot. Capacity is a matter of wallets and keys; nothing in the contracts changes.

Units. A unit error does not throw; the numbers just come out wrong. All conversions live in `Backend/src/lib/units.ts` and `Backend/src/venue/perpl/units.ts`, integers only, with tests; `PerplReader` does the same maths on-chain.

Quantity	Perpl	Laxu contracts
Price	integer × 10^priceDecimals (ETH: 2)	× 1e18
Size	lots × 10^lotDecimals (ETH: 3)	base quantity × 1e6, always positive
AUSD amounts	base units, 6 decimals	6 decimals (shares too)
Leverage	`lv` = leverage × 100	whole number, 1–20
Funding rate	micros per interval; positive = longs pay shorts	—
Accrued funding	—	signed AUSD, 6 decimals, positive = received

Example: 14 ETH lots (lot decimals 3) is size $14 \times 10^6 / 10^3 = 14{,}000$; on a $200 move, PnL = 14,000 × 200e18 / 1e18 = 2,800,000 = $2.80.

---

4. Mechanism design

4.1 PositionToken: one contract per trade

Each trade is an **EIP-1167 clone** of one `PositionToken` implementation, an ERC-20 and ERC-7540 vault with its own market, direction, leverage, entry, size and capital.

Only real trades get tokens. At creation the token reads the slot's Perpl position through `PerplReader`: direction and size must match exactly, and entry within 0.5% (`ENTRY_TOLERANCE_BPS = 50`).

Valuation:

```
value = capital + size × (mark − entry) / 1e18   (negated for shorts)
              + (fundingAccrued − fundingSettled)
NAV per token = value / totalSupply
```

- **The mark** is read by `currentMark()` through `PerplReader`, which marks it invalid when the market is paused, the exchange is halted, or Perpl's own price age has passed. If the read fails, the last cached mark is used, so valuation never reverts.
- **Funding** arrives through `applyFunding`; Perpl resets its funding accumulator when a position grows, so it cannot be read on-chain. The backend derives it as venue total minus capital minus price PnL plus funding settled, so it absorbs fees too.

There is no function that lets the operator set a price.

Price rules. `isPriceFresh()` is true for a closed position, or when the last funding report is within `FUNDING_MAX_AGE` (2 hours) **and** the Perpl mark is live, valid and within `MARK_MAX_AGE` (5 minutes). `venueDrift()` shows the token's size and entry next to Perpl's.

Asynchronous requests (ERC-7540). Buy-ins (`requestDeposit`) and redeems (`requestRedeem`) need a Perpl order first, so the operator fulfils them afterwards. Fulfilment settles in the same transaction at the live NAV. If a request is not fulfilled within **20 minutes** (`REQUEST_CANCEL_TIMEOUT`), the requester can cancel it: a buy-in returns its AUSD (the 2% fee stays with the creator), a redeem re-mints its shares.

Proportional accounting. With value `V` and supply `S`, a buy-in of `a` mints `a × S / V` shares, adds the size actually filled on Perpl, moves entry to the size-weighted average, and adds `a` to capital. A redeem of `f = shares / S` pays `f × V` and scales capital and settled funding by `f`. Both keep **NAV per token equal for every holder**; leverage stays constant unless the change is below Perpl's minimum order size. When the last share leaves, the position closes and settles at zero by itself.

Lifecycle:

- `list(nickname)` is creator-only and one-way; nicknames are at most 32 bytes.

- **Buy-in fee:** 2% (`BUY_IN_FEE_BPS = 200`) of each non-creator buy-in, to the creator; the creator's own top-ups are free.
- `requestClose` needs the creator to hold 100% of supply with no buy-in pending.
- **Close → settle → claim.** `close()` fixes an estimate at the live mark plus final funding. The operator withdraws from Perpl and calls `settle(assets)` with what was recovered; the contract checks it holds that plus pending buy-ins. Holders claim pro rata; `claimFor` always pays the holder. A Perpl liquidation is recorded as `close(…, true)`.

Per-holder stop-loss and take-profit. The creator's levels are everyone's default; each holder can set or clear their own. When a level is crossed at the live Perpl mark, the operator exits only that holder's wallet balance at NAV; the contract checks the level itself. Tokens posted as collateral are never touched.

4.2 Lending: a shared vault with isolated pools

LendingVault (ERC-4626). Lenders deposit AUSD once. Lent-out AUSD still counts in `totalAssets`, so borrowing does not move the share price and interest raises it. Only registered pools can draw, each up to its own debt ceiling (10,000 AUSD per new pool).

LendingPool, one per token, from a permissionless factory. Risk parameters come from the token's own leverage; **the caller sets none of them**.

Leverage	Max LTV	Liquidation threshold	Liquidator bonus
1–5×	50%	60%	8%
6–10×	40%	50%	10%
11–20×	25%	35%	12%

Higher leverage borrows less and pays liquidators more. Not back-tested.

Health factor = collateral value × threshold / debt, read live; 1.2 at the 1–5× LTV cap.

Liquidation is permissionless and never paused. The close factor is 50% of debt per call, 100% when health is below 0.95 or collateral is under 50 AUSD (dust). The bonus is paid in position tokens, capped at the collateral held. Example: on 100 AUSD of debt, a liquidator repays 50 and receives 54 AUSD of tokens.

Interest is a flat 10% APR, simple: liquidity is shared, so one pool's utilization is no rate signal.

Limits: bad debt is not absorbed and there is no insurance fund. The tier table has no upper bound of its own; the factory's 20× limit is the bound.

4.3 Price freshness: an asymmetric rule

The mark is read on-chain from Perpl on every valuation; nothing pushes it. The reporter runs every minute: it pushes funding when the target moves by max(0.10 AUSD, 0.1% of capital) or after 30 minutes (the heartbeat), and executes any stop-loss or take-profit the Perpl mark has crossed.

- **Stale data blocks new risk:** `borrow()` and `withdrawCollateral()`.
- **Stale data never blocks `liquidate()`, `repay()` or `healthFactor()`.** Liquidating on last-known data beats leaving bad debt open.
- **Closed positions are exempt**, so borrowers can always withdraw collateral to claim.

With the backend down, borrowing still works for up to 2 hours. The mark keeps coming from Perpl; only the funding report ages. Borrowing pauses when it passes `FUNDING_MAX_AGE`, so 90 to 120 minutes after the backend stops, given the 30-minute heartbeat. The recorded run borrowed every two minutes for ten minutes with the backend off; a contract test covers one hour. If Perpl's mark itself goes stale, borrowing pauses too.

What the operator key can and cannot do.

- **It supplies:** trade details at creation (checked against Perpl); funding (timestamps strictly increasing, at most 60 seconds ahead); fill size and price on fulfils and triggers; final funding at `close()`; the recovered amount at `settle()`.
- **It cannot:** set a price; mint a token for a trade Perpl does not hold; mint itself shares; exit a holder whose level is not crossed at the live mark; close a position without the creator's request, except to record a liquidation; pay a claim to anyone but the holder; touch pending buy-ins or unclaimed settlement funds; block `repay()`, `liquidate()` or `claim()`.

Limit: freshness guards against *old* data, not *wrong* data. A faulty operator could misstate funding, a fill or a settlement amount (§6).

4.4 Loan protection

Why a signer. `repay` pays the caller's debt from the caller's wallet, so Laxu must act as the user. It does so with a **Privy signer** (a server key the user adds to their embedded wallet) bounded by a **policy**.

Who can use it. Embedded-wallet (email sign-in) users only; a plain external wallet cannot hold a Privy signer.

Setup. The user sets a trigger health (default 1.15, range 1.05–3.0), a target (default 1.30, at least trigger + 0.10) and a maximum spend (up to 500 AUSD). Then:

1. They add Laxu's signer with the policy built for this loan.
2. They approve an AUSD allowance to the pool **equal to the maximum spend**, not unlimited.

The backend activates the rule only after checking both, with Privy and on-chain.

The policy has one `ALLOW` rule for `eth_sendTransaction`: target is this pool, chain is 10143, value is 0, function is `repay`, and `amount <= maxPerCall` (half the maximum spend). Everything else is denied by default. There is deliberately no `DENY` rule: on Privy a matching `DENY` overrides any `ALLOW`. The policy limits *what* Laxu can sign; the allowance limits *how much* in total.

The worker checks every enabled rule **every 5 seconds**. At or below the trigger, and outside a 60-second cooldown, it computes the debt that sits on the target:

```
D' = collateralValue × thresholdBps × 1e18 / (10_000 × target)
repay = (debt − D') + 1% buffer
```

The amount is clamped by the debt, the per-call cap, the remaining spend, the wallet's AUSD and the allowance; under 0.01 AUSD nothing is sent. Privy signs and broadcasts from the user's wallet, which **pays the MON gas**. With too little MON, the worker skips and tells the user.

Off: disable in the app (the backend stops at once), remove the signer, set the allowance to 0.

Evidence (`docs/privy-findings.md`, Monad testnet): after the user added the signer, Privy showed it with its policy (`ours present: true`). Under a test policy capped at 1,000, `approve(1000)` from the user's wallet was mined; `approve(1001)`, a plain `transfer`, and a call to another address were each refused with `policy_violation`. After removal, the same call failed with HTTP 401. A server-side QA run passed 22 of 22 checks. **Not yet run:** a full testnet cycle in which a real loan crosses its trigger and is repaid, and the browser walk-through.

What it does not do. It cannot withdraw collateral, borrow, transfer AUSD, or touch other tokens or pools. It cannot help a wallet with no AUSD or no MON. One wallet protects one loan at a time. A very fast move can still liquidate between checks.

---

5. The invariants we hold

Test names are quoted exactly from the test files.

Invariant	Enforced by	Test
A buy-in never changes NAV	Proportional mint	*"a buy-in at NAV 1.48 leaves NAV at 1.48 and grows size, entry and capital"*
A redeem pays its share and keeps NAV	Fraction-scaled accounting	*"redeeming 50% pays 50% of totalAssets, halves capital and effective funding, and keeps NAV"*

Invariant	Enforced by	Test
The mark is read on-chain; no operator price	`currentMark()` via `PerplReader`	*"operator has no function that sets the mark price"*; *"NAV follows the venue mark with no operator transaction"*
Tokens exist only for real Perpl trades	Creation check	*"createPosition reverts on size mismatch"*; *"createPosition reverts when entry differs from the venue by more than 0.5%"*
Funding reports only move forward	`applyFunding` checks	*"rejects a stale (non-increasing) or future funding timestamp"*
Profit raises the limit with no transaction	Live valuation	*"hands the borrower more headroom automatically when the position gains value"*
Stale data cannot create debt	`freshOracle`	*"blocks borrow() once the venue mark is older than MARK_MAX_AGE"*; *"borrow reverts when funding is older than FUNDING_MAX_AGE"*
Borrowing works while the reporter is down	Mark from Perpl, `FUNDING_MAX_AGE`	*"borrow keeps working for 1 hour with no operator transaction while the venue mark updates"*
Stale data never blocks liquidation	Asymmetry	*"does NOT block liquidate() on stale data -- liquidating on last-known data beats not liquidating at all"*; *"liquidation still works when the venue read reverts"*
Liquidation is bounded and permissionless	Close factor, fixed bonus	*"liquidates at 50% close factor and pays the 8% bonus in shares"*; *"is permissionless -- an address holding no protocol role can liquidate"*
Closed positions never trap collateral	Exemption	*"exempts a closed position: repay + withdrawCollateral work long after close, borrow does not"*
Payouts stay fair until the last claim	Constant rate during claims	500/300/200 claim test, `PositionToken.js:1042`
Triggers cannot pull collateral	Wallet balance only	*"only exits the wallet balance, not tokens posted as LendingPool collateral"*
Leverage stays within the tiers	`MAX_LEVERAGE = 20`	*"factory rejects leverage above 20"*
The signer can only `repay` one pool	Privy policy	*"repay policy: one ALLOW rule, no DENY (a DENY-all would override the ALLOW), and the ABI is repay only"*
Protection never overpays	`planRepay` clamps	*"never more than the debt"*; *"each clamp binds on its own, and is named"*

Totals: 143 contract tests pass (76 `PositionToken`, 38 lending, 29 Perpl integration and trust model), plus 2 pending tests that need a live Monad RPC. **152 backend tests** pass.

```
cd Contracts && npx hardhat test
cd Backend && npm test
```

---

6. Trust model and tradeoffs

Component	Today (testnet)	Why	Path to remove trust
Perpl positions	**Custodial:** Laxu's slot wallets hold the accounts	One wallet = one account	Per-user delegated accounts (`target_profile`); the owner-side grant is undocumented: **planned, pending exchange support**
Price	Mark read on-chain via `PerplReader`; funding and fills are the trusted inputs	Perpl does not expose cumulative funding on-chain	Second reporter or on-chain checks **(planned)**
Fills and settlement	Supplied by the backend; contract checks the trade at creation and that payouts are funded	Orders and withdrawals go through Perpl	On-chain fill checks **(planned)**
Liveness	One backend	Speed of shipping	Multiple reporters **(planned)**
Lending	**On-chain**, permissionless liquidation	Lenders' money sits here	—
Capacity	5 concurrent positions today, one per provisioned slot	One wallet = one account	Provision more wallets and keys; nothing in the contracts changes
Protection signer	Privy authorization key, bounded to `repay` by policy and allowance	Repaying means signing as the user	Smart accounts or session keys **(planned)**

When the backend fails: repay and liquidation stay open; borrowing works while funding is within `FUNDING_MAX_AGE`; closed positions' collateral can always be withdrawn; buy-ins and redeems can be cancelled after 20 minutes; interrupted opens resume or refund; protection stops acting. Settlement after a close needs the backend. **Not handled:** a Perpl outage longer than the timeouts during an open needs manual resolution.

Known Perpl-side gap. The testnet closes trading sockets with `1008 ping timeout` several times an hour, though Laxu answers every ping at once; the evidence points to the server. The client reconnects with backoff (1 s up to 60 s), and interrupted orders are decided by the lot rule (§3.4).

---

7. Decisions and what we cut

Borrowing, not copy trading. Lending against an open trade is the unmet need; buy-ins stay because buyers can borrow too.

A token, not a platform loan. Lending from Laxu's balance sheet would make Laxu a bank. A token puts collateral on-chain: anyone can lend or liquidate. *Cost:* an off-chain leg and an on-chain leg that must agree (§3.4).

Isolated pools. One bad position cannot hurt another pool. *Cost:* no cross-collateral borrowing.

Reading the mark on-chain rather than trusting a pushed price. The operator has no price input, and borrowing survives a backend outage. *Cost:* funding still needs a reporter, and every token depends on `PerplReader`.

Privy signers with a policy rather than a custodial server wallet for protection. The user's AUSD stays in the user's wallet, and the signing key can only call `repay` within a cap. A Privy server-wallet spike (`Backend/scripts/privy/01-server-wallet.ts`) shows Privy can sign and broadcast on Monad testnet; it is evidence, not a shipped path. *Cost:* embedded wallets only, one loan per wallet.

What we cut:

- **Liquidation bot:** built, off by default; `liquidate()` is open to anyone.
- **Vault screen:** lenders use the ERC-4626 vault directly.
- **Secondary market:** later; tokens are ERC-20s.
- **Community feed, strategy bot:** not built.
- **General Perpl order entry:** trades start through the open flow and change only by buy-in, redeem, trigger and close.

All cut to ship **open → borrow** end to end.

---

8. Market and positioning

Three adjacent categories:

- **Perp venues** (GMX, Hyperliquid): a trade cannot be collateral until it closes.
- **Pooled liquidity tokens** (GMX GM): they tokenize the house's side, not the trader's.
- **Leveraged tokens** (Toros, Index Coop): platform-defined, with no lending against them.

	Cannot borrow against it	**Can borrow against it**
Platform-made product	Leveraged tokens (Toros, Index Coop)	Pooled LP tokens (GMX GM on Dolomite)
Your own trade	Perp venues (GMX, Hyperliquid)	**Laxu**

Evidence the pattern works. GMX GM tokens, minted request-then-execute, are isolated collateral on Dolomite. Laxu applies that to the trader's side.

Expansion: other perp venues. The token and pool contracts do not depend on Perpl; only the backend adapter and `PerplReader` do.

---

9. Go-to-market

- **Start with** Perpl traders on Monad testnet, then mainnet.
- **Grow through** shareable trade pages and the 2% creator fee.
- **Cold start:** Laxu seeds the vault. **Launch safely:** low debt ceilings, raised over time.
- **Channels:** the Monad builder community, the hackathon, X.
- **Perpl:** Laxu integrates through Perpl's public API and exchange contract. Through the hackathon mentors we asked Perpl about whitelisting our origin and issuing a builder code. Perpl pointed us to its builder program, which covers both; we have submitted the registration and have no reply yet. There is no partnership.

---

10. Business model

Today: the 2% buy-in fee goes to creators and interest to lenders. **The protocol earns nothing.** The builder fee is 0.

(Planned):

- A **reserve factor** on interest, also funding a bad-debt reserve.
- A **Perpl builder-code fee** as an option: Perpl issues an id (1–255) and orders carry `bf` in hundred-thousandths, at most 100 (0.1%). It is built behind a flag that is off.
- A **split buy-in fee or profit share**.

---

11. Roadmap

1. Funding and fills verified on-chain or by a second reporter; multiple reporters.
2. **User-owned Perpl accounts (less custody).** Perpl's answer is builder-bound API keys: Laxu registers as a builder, the user signs one EIP-712 enrolment with their own wallet, and Laxu trades their account through a key it can only use within Perpl's scopes and our fee ceiling. The code for this exists for slot keys (`venue/perpl/enrollment.ts`, `builder.ts`) and waits on Perpl issuing a builder id and whitelisting our origin. It fits plain trading and following, not borrowing: a user can close the position, withdraw or revoke the key at any time, so a loan backed by it could not be liquidated reliably. Positions that back a loan therefore stay in Laxu slots, where liquidation is enforced.

3. Loan protection for external wallets (smart-account or session-key route).
4. Secondary market and social layer.
5. Provision more slots as usage grows.
6. More venues (adapter swap).
7. Mainnet: audit, debt caps, legal review.

---

12. Risks and disclosures

- **Leverage:** positions can lose all their capital quickly.
- **Liquidation:** below health 1, anyone can liquidate, at an 8–12% bonus from the borrower's collateral.
- **Venue:** Perpl outages, rule changes or testnet resets affect every position.
- **Oracle and reporter:** the mark is Perpl's; funding, fills and settlement amounts come from one trusted backend (§6).
- **Smart contracts: unaudited**.
- **Loan protection:** a policy mistake, a compromised signer key or a worker bug can still spend up to the user's allowance, on repayments of that one loan only.
- **Regulatory:** tokenized leveraged positions may be derivatives in some places; legal review precedes mainnet.

Testnet only, no real funds. Laxu is an independent project built on Perpl, not affiliated with or endorsed by Perpl.

---

13. Credits and how it was built

- **OpenZeppelin Contracts:** ERC-20, ERC-4626, Clones, SafeERC20, access control, reentrancy guard.
- **OpenZeppelin community contracts:** `ERC7540`, `ERC7540AdminDeposit`, `ERC7540AdminRedeem`, **vendored and modified** in `Contracts/contracts/vendor/`. Four storage fields (`_deposits`, `_redeems`, `_totalPendingDepositAssets`, `_totalPendingRedeemShares`) are `internal` rather than upstream's `private`, so `PositionToken` can settle on fulfil and cancel requests. Each header records the change and the upstream file's sha256.
- **Also:** Perpl's API, WebSocket and exchange contract; Privy (auth, embedded wallets, signers, policies); viem; Prisma with Neon Postgres; Next.js; TradingView Lightweight Charts (Apache 2.0).

Where the key decisions live:

File	Decision
`Contracts/contracts/PositionToken.sol`	Valuation, creation check, freshness, buy-in/redeem, close/settle/claim, triggers
`Contracts/contracts/PerplReader.sol`	Perpl mark and positions, unit conversion

File	Decision
`Contracts/contracts/LendingPool.sol`	Risk tiers, `freshOracle`, liquidation, interest
`Contracts/contracts/LendingVault.sol`	Shared liquidity, debt ceilings
`Backend/src/services/openPosition.ts`	Resumable open flow, refunds
`Backend/src/services/venueOrders.ts`	Outcome-unknown rule
`Backend/src/services/reporter.ts`	Funding reconciliation and push policy
`Backend/src/services/protection.ts`, `protectionMath.ts`	Protection worker, repay maths
`Backend/src/lib/units.ts`, `Backend/src/venue/perpl/units.ts`	Unit conversions

How it was built. One developer with an AI coding assistant (Claude Code). The product decisions, architecture, risk parameters and tradeoffs are the author's. All code was reviewed and tested, and corrected where wrong. One example: Perpl's funding `rate` is in micros per interval, but the app divided it as hundred-thousandths, so it showed funding ten times too high (BTC 0.0100% instead of 0.0010%). Checking the rate against Perpl's payment-per-lot figure (`ppl` = index × rate / 10^6) on all eight markets exposed it, and the app reads micros.

Appendix

A. Parameters

Parameter	Value	Where
LTV / threshold / bonus, 1–5×	50% / 60% / 8%	`LendingPool._riskTierFor`
LTV / threshold / bonus, 6–10×	40% / 50% / 10%	`LendingPool._riskTierFor`
LTV / threshold / bonus, 11–20×	25% / 35% / 12%	`LendingPool._riskTierFor`
Close factor	50%; 100% below health 0.95	`CLOSE_FACTOR_BPS`, `CLOSE_FACTOR_HF_THRESHOLD`
Borrow rate	10% APR, simple	`BORROW_APR_BPS = 1_000`
Debt ceiling per new pool	10,000 AUSD	`LendingPoolFactory.defaultDebtCeiling`
Leverage range	1–20×	`PositionTokenFactory.MAX_LEVERAGE`
Per-market cap	min(20, `initial_margin` / 100)	`venue/perpl/units.ts`
Mark max age	5 minutes	`MARK_MAX_AGE`

Parameter	Value	Where
Funding max age	2 hours	`FUNDING_MAX_AGE`
Entry tolerance	0.5%	`ENTRY_TOLERANCE_BPS = 50`
Buy-in fee	2%, to the creator	`BUY_IN_FEE_BPS = 200`
Cancel window	20 minutes	`REQUEST_CANCEL_TIMEOUT`
Reporter tick	60 seconds	`REPORTER_INTERVAL_MS`
Funding push	max(0.10 AUSD, 0.1% of capital)	`FUNDING_PUSH_MIN`, `FUNDING_PUSH_BPS`
Funding heartbeat	30 minutes	`FUNDING_HEARTBEAT_SECONDS`
Order attempts	3	`MAX_ATTEMPTS`
Slippage bound	1%, clamped per market	`PERPL_SLIPPAGE_BPS`
Reservation timeout	15 minutes	`RESERVATION_TIMEOUT_MS`
Slot reserve	100 AUSD	Perpl minimum account open
Slots provisioned	5 (an operator setting)	backend database
Protection interval / cooldown	5 s / 60 s	`PROTECTION_INTERVAL_MS`, `PROTECTION_COOLDOWN_S`
Protection max spend	500 AUSD	`PROTECTION_MAX_SPEND_CAP`
RPC pacing	12 req/s; logs in 100-block windows	`RPC_MAX_RPS`, `RPC_LOGS_MAX_RANGE`

B. Deployed addresses

Monad testnet, chain ID 10143. Explorer: https://testnet.monadvision.com

Contract	Address
PerplReader	`0xF880353b3DE09ca7684e63cc1Ec3f5Ef33599e08`
PositionToken (implementation)	`0x96ddDA169839B83Ac13B452B5BDfF35B9c61F94d`
PositionTokenFactory	`0x2df05E826b70a2719a2828fFE9Dc25FB65CfA690`
LendingVault	`0x2805D873Acea8441E1411040Af4FCd28DcDEc1a3`
LendingPool (implementation)	`0x1ad132b698f3B8839414A652ed5ab14bAf1211E5`
LendingPoolFactory	`0x111255949cb82ca42927f6537047398c3d32B3c8`

Perpl's contracts (external):

Contract	Address
Perpl Exchange	`0x1964C32f0bE608E7D29302AFF5E61268E72080cc`
AUSD	`0xa9012a055bd4e0eDfF8Ce09f960291C09D5322dC`

Contract	Address
AUSD faucet	`0xd236c18D274E54FAccC3dd9DDA4b27965a73ee6C`

C. Glossary

- **Perpl:** the perpetual futures exchange on Monad that Laxu trades on.
- **AUSD:** the stablecoin Perpl uses as collateral; Laxu lends and settles in it.
- **Mark price:** the exchange's reference price.
- **Funding:** periodic payments between longs and shorts.
- **Leverage:** notional divided by capital.
- **Health factor:** collateral value × liquidation threshold / debt; below 1, liquidatable.
- **ERC-4626 / ERC-7540:** tokenized vaults, and their asynchronous request-then-fulfil extension.
- **Slot:** a Laxu-held wallet that owns one Perpl account and its trade-scope key, holding one open trade at a time.
- **Signer:** a server key added to a Privy embedded wallet.
- **Policy:** what a Privy signer may sign.
- **Builder code:** a Perpl id for integrator attribution and a capped fee.